

Real-world (convert a real function to async) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
from openai import AsyncOpenAI
import asyncio

client = AsyncOpenAI()

async def ask(question):
    response = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": question}],
    )
    return response.choices[0].message.content

answer = asyncio.run(ask("What is 2 + 2?"))
print(answer)
```

Expected output:

```
2 + 2 equals 4.
```

The only real changes from the sync Doc04 starting point: `AsyncOpenAI` instead of `OpenAI` , `async def` on the function, `await` in front of the API call, and `asyncio.run(...)` instead of calling `ask(...)` directly. Nothing about the model, the messages, or the return value changed at all.

This version works correctly. It's missing type hints and an `if __name__ == "__main__": guard` — both fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, with a `main()` and a `__main__` guard

```
from openai import AsyncOpenAI
import asyncio

client = AsyncOpenAI()

async def ask(question: str) -> str:
    response = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": question}],
    )
    return response.choices[0].message.content

def main() -> None:
    answer = asyncio.run(ask("What is 2 + 2?"))
    print(answer)

if __name__ == "__main__":
    main()
```

Expected output:

```
2 + 2 equals 4.
```

Difference from Basic: the `-> str` type hint on `ask` documents exactly what the coroutine resolves to once awaited, same as the sync version would have had. The `if __name__ == "__main__":` guard, calling a plain `main()` that itself calls `asyncio.run(...)`, means this file can be imported elsewhere (a test, or a FastAPI route in Doc12) and reuse `ask()` directly with its own `await`, without accidentally triggering a second, nested event loop the way calling `asyncio.run()` again from inside an already-running one would (this is the exact `RuntimeError` covered in `first_coroutine`'s Advanced Hint).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a client-level timeout, and the SDK's specific timeout error

```
from openai import AsyncOpenAI, APITimeoutError
import asyncio

client = AsyncOpenAI(timeout=30.0)

async def ask(question: str) -> str:
    response = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": question}],
    )
    return response.choices[0].message.content

async def main() -> None:
    try:
        answer = await ask("What is 2 + 2?")
        print(answer)
    except APITimeoutError:
        print("the API call took too long and was cancelled")

if __name__ == "__main__":
    asyncio.run(main())
```

Expected output (normal case):

```
2 + 2 equals 4.
```

Expected output if OpenAI's servers genuinely hang past 30 seconds:

```
the API call took too long and was cancelled
```

`timeout=30.0` on the client is a default every call made through it inherits — no individual caller has to remember to wrap their own call in `asyncio.wait_for(...)`. `APITimeoutError` is the SDK's specific exception for this — catching it instead of a bare `except Exception:` means a real bug elsewhere in `ask()` (a typo in `model=`, say) still surfaces as its own clear error instead of being mistaken for a slow API call.

Approach 2 — used from inside an already-async caller (a FastAPI route), the way it'll actually be used in Doc12

```
from openai import AsyncOpenAI, APITimeoutError

client = AsyncOpenAI(timeout=30.0)

async def ask(question: str) -> str:
    response = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": question}],
    )
    return response.choices[0].message.content

# a stand-in for Doc12's FastAPI route -- no asyncio.run() anywhere in here,
# because the web framework already owns and is running the event loop
async def handle_question_request(question: str) -> dict:
    try:
        answer = await ask(question)
        return {"answer": answer}
    except APITimeoutError:
        return {"error": "The model took too long to respond. Please try again."}
```

Expected output, if you call `asyncio.run(handle_question_request("What is 2 + 2?"))` from a script to simulate what the framework would do:

```
{'answer': '2 + 2 equals 4.'}
```

Notice `ask()` itself is completely unchanged from Approach 1 — only the caller changed.

`handle_question_request` never calls `asyncio.run()` because, in real Doc12 code, FastAPI already started and owns the event loop before your route function is ever called; it just `await s` `ask()` directly and turns a timeout into a clean error response instead of a crashed request.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's `ask()` has no timeout and is only ever called from a script's own `asyncio.run()`. Approach 1 adds the timeout and shows what a caller does with it in that same script-style setup. Approach 2

keeps the exact same `ask()` function, unmodified, and shows the *other* real shape it'll be called in — from inside a framework's already-running event loop, where `asyncio.run()` would be the wrong (and broken) thing to write.

Which one should you actually use? Ship Approach 1's client-level timeout on every `AsyncOpenAI()` (or `OpenAI()`) client you create — it's one keyword argument, set once, that protects every call made through that client for the rest of the program's life. Approach 2's shape — `ask()` staying a plain, reusable coroutine with no `asyncio.run()` inside it — is exactly what this conversion needs to already look like before Doc12, since that's the literal function you'll import into a FastAPI route; writing it that way now means Doc12 is a five-minute wiring job instead of a second conversion.